Augur: Measuring When Predicting Data Makes It Faster to Move

Peter Hansen
peterhansen6002@gmail.com

Abstract*—Can a model that predicts data make it faster to store and transfer? We study two forms of prediction under one framework: predicting *which item is requested next*, so it can be prefetched into a bounded cache, and predicting *the content of the data itself*, so that sender and receiver share a model and only the surprises are sent. We built AUGUR, an open-source, user-space testbed with a byte-bounded cache, sequential, stride and Markov predictors, a confidence-gated prefetch scheduler, a trace-driven storage simulator, and an exact, integer-only context-mixing codec (augur-cm). On ten real files, augur-cm was smallest on six of the seven compressible files, 6.8–31.5 % smaller than the best of gzip, bzip2, xz, zstd and brotli, and 5.2 % larger than xz on one executable. Every output decoded byte for byte, and incompressible input cost one extra byte. The price is speed: augur-cm decodes at about 0.5 MB/s, over a thousand times slower than zstd. On a link, it is the fastest option only below roughly 0.04–0.11 Mbit/s, and it would need 16–53× faster decoding to win at 2 Mbit/s. In simulated storage workloads, history-based prefetching helped only where access history repeated (1.08–1.10× for the default predictor), while trusting single observations slowed workloads to as little as 0.36×. Replaying the workloads with real reads on a hard disk with a cold page cache, a learned history-based predictor never beat a plain LRU cache by more than 1.03× (fixed-order epochs) and ran at 0.94× on recurring routines, where it read 28 % more data; a clairvoyant predictor reached 1.21–1.31× on the non-sequential workloads, which bounds what access prediction could gain on that drive. Prediction therefore saves bytes reliably but saves time only when its compute and misprediction costs are small compared with the time the bytes would take to move. We release code, corpus manifest and all measurements.

***Index Terms*—prefetching, lossless compression, context mixing, learned caching, data transfer, benchmarking**

I. Introduction

Computers spend much of their time waiting for data: a program stalls on a disk read, a download crawls over a slow connection, a training job idles while its next batch is fetched. A tempting idea is to borrow from graphics, where learned models reconstruct detail that was never rendered, and apply it to data in general: if a model can predict what will be needed, or even what the data *is*, less has to be waited for.

This paper asks whether that is true, and how much, by measurement. We do not assume that machine learning beats existing techniques, and we do not claim the underlying ideas are new: prefetching, learned caching and model-based compression are all established (Section II). Our contribution is a small, reproducible testbed and an honest accounting of cost and benefit. Specifically:

- A common framing in which one predictor of "what comes next" serves two consumers: a prefetch scheduler (Section III-A) and an arithmetic coder (Section III-D).
- An exact predictive codec whose model is integer-only, so encoder and decoder agree on any machine, with a byte-for-byte round-trip check on every measurement.
- Measurements on real files of compression ratio, speed and ablations against five widely used codecs (Section V).
- A closed-form model, driven by those measurements, of when coding beats sending raw bytes, and of how much faster the codec would have to be to win (Section V-C).
- A simulated study of prefetching that records when prediction helps, hurts, or wastes bandwidth (Section V-E).
- A real-I/O replay of the same workloads on a hard disk with cold-cache control, OS-only baselines, and CPU, memory and device-read measurements, which tests the simulator's conclusions and shows where it was wrong (Sections III-C and V-F).

II. Background and Related Work

Access prediction. Predictive prefetching for the Web, driven by server-side access history, dates to Padmanabhan and Mogul [1]. For storage, MITHRIL mines block associations from request history alone [2]. Deep-learning input pipelines are a current target: NoPFS exploits the fact that a seeded shuffle makes the access order computable in advance [3], CoorDL analyzes and reduces data stalls [4], and FFCV removes loader bottlenecks with a faster format and loader [5]. Learned policies are also used for caching: LRB approximates Belady's optimal eviction with a learned model for CDN caches [6], and Baleen applies learned admission and prefetching to flash caches [7]. At the CPU level, Voyager [8] and Pythia [9] use neural and reinforcement-learning prefetchers, and KML places machine learning inside the operating system to tune readahead [10].

A caution comes from deployment. HTTP/2 server push let a server send what it expected a client to need. Chrome's data showed that fewer than 40 % of received pushes were used and that studies found little or no benefit; Chrome proposed removing it [11]. Wrong predictions cost bandwidth, which is the effect our simulator also exposes.

Predictive coding. A model that predicts data is a compressor: Shannon's entropy bound [12] is reached by coding each symbol with its predicted probability, which arithmetic coding does efficiently [13]. The PAQ family combines many context models with a learned mixer [14], and large neural models now

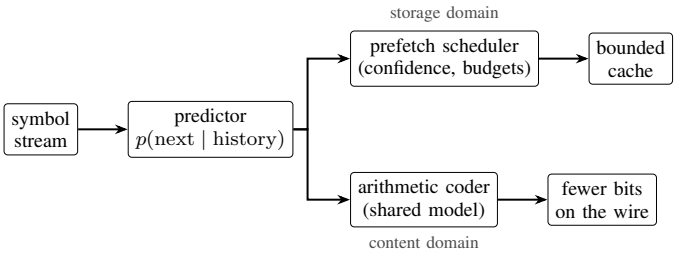


Fig. 1. One predictor feeds two consumers. The decoder in the content domain runs the same predictor on the data decoded so far, so only the model's surprises are transmitted.

rival or beat it: NNCP uses a Transformer and reports a smaller total than cmix on a 1 GB English text benchmark [15], and a recent study shows that a language model trained mainly on text compresses image patches to 43.4 % and audio to 16.4 % of raw size, ahead of PNG and FLAC [16]. Production systems use fast general codecs instead: zstd [17] and brotli [18] trade ratio for very fast decoding. That trade-off is the subject of this paper.

III. Method

A. One predictor, two consumers

Both uses share one core: a model that observes a stream of symbols and returns a probability for the next one (Fig. 1). In the *storage* domain a symbol is an item identifier (a block, file or dataset sample), and a scheduler turns high-confidence predictions into prefetch reads under budgets for bytes and reads in flight. In the *content* domain a symbol is a byte, and an arithmetic coder turns the probability into bits. A resource model supplies the cost of each action: seek time and bandwidth for a disk, round-trip time and bandwidth for a link.

B. Storage testbed

The testbed has five modules: a byte-bounded cache (LRU or LFU, never exceeding its byte budget, returning stored values untouched), predictors, a scheduler, a trace-driven simulator, and a benchmark runner. Predictors are a sequential readahead, a stride detector, an order-k Markov model with back-off (confidence is the product of transition probabilities along the predicted chain), and a clairvoyant oracle that reads the future from the trace as an upper bound. Frequency-based prediction is covered by LFU eviction rather than as a prefetcher, because it would mostly name items already cached. The scheduler acts only above a confidence threshold (0.4) and within caps on reads in flight (8) and bytes in flight (8 MiB), and can back off when recent prefetches were wasted. The simulator models one FIFO storage device, so prefetch reads queue ahead of later demand reads, and a request for an item whose prefetch is still in flight waits only for the remainder (a late hit). Device parameters are nominal assumptions, not measurements: HDD 8 ms and 150 MB/s, SATA SSD 0.1 ms and 500 MB/s, NVMe 0.08 ms and 3 GB/s. Seven seeded workloads cover sequential, random, Zipf working-set, phased, recurring-routine and deep-learning epoch access, the last with and without reshuffling. The routines workload is our own construction to test the case where history repeats.

C. Real-I/O replay

The same traces, predictors and scheduler drive real reads. Each workload gets a file holding its items back to back; every item begins with its own identifier, checked on every read. A run starts by evicting the whole file from the page cache with `posix_fadvise(DONTNEED)`. Because that call is advisory, the harness first times cold and warm random reads on the file and warns if the ratio is below 1.5. The consumer sleeps 1 ms between requests (the application's compute), serves each request through the cache logic, and hands prefetch reads to four worker threads; all cache bookkeeping stays on the consumer thread. A dataset larger than memory is emulated by dropping an item's pages from the kernel page cache when the application cache evicts it (for the OS-only configurations, when it falls out of an LRU ledger of the same byte size), so kernel and application hold the same items; kernel readahead stays enabled. Two OS-only baselines read with no application cache: default kernel behaviour (`os-default`) and readahead disabled (`os-noreadahead`). We record per-read latency, end-to-end time, the bytes the kernel fetched from the device (`/proc/self/io`), process CPU time, peak resident memory, and predictor time separately. Configuration order is shuffled within each repeat.

A shared machine occasionally stalls the drive, so we call a run *disturbed* if it took more than 1.3 times the fastest repeat of its cell, exclude it, and compare each configuration with LRU repeat by repeat, using only repeats where both runs were undisturbed.

A caution from our first design. Our first version dropped each item's pages immediately after the application read it. That made every application-cache configuration slower than the OS-only baseline by 1.2× on the image-sized workloads and 1.06× on recurring routines (three repeats; 85.1 s versus 69.4 s in a diagnostic run), probably by disturbing kernel readahead, though we did not establish the mechanism. Evicting from the kernel only when the application cache evicts removed the gap (68.9 s in the same diagnostic run). The design described above is the corrected one; results from the first design are kept in the repository.

D. Exact predictive codec

augur-cm is a deliberately small design in the lpaq lineage [14]. A bitwise model predicts each bit from a direct-indexed context of the previous byte, hashed contexts of the previous 1, 2, 3, 4, 5, 6, 8 and 12 bytes, and a match model that follows the longest recent repeat. Each context slot holds an adaptive probability. A gated logistic mixer, with one weight set per partial byte, combines the stretched probabilities. A 32-bit binary arithmetic coder encodes each bit with the mixed probability [13].

Exactness. All arithmetic is integer: probabilities are 12-bit, the logistic and its inverse are table lookups built from

integers, and the mixer uses fixed-point weights. No floating point is used anywhere, so encoder and decoder compute identical probabilities on any machine, which a model shared between two computers requires and which floating-point neural models do not give by default. The container is one mode byte followed by either the model stream (an 8-byte length and the coded body) or the raw input. If the model stream is not smaller than the input, the encoder stores the input raw. This fallback is needed: on 50 000 random bytes the bare model stream (small tables, in a unit test) was 3.5 % larger than the input, because high-order contexts learn noise. A decoder never reads past its input or writes past its declared output, and corrupt streams yield wrong bytes or an error, not a crash (tested by bit-flip fuzzing).

Tuning. The mixer learning rate and counter limit were chosen by grid search on a separate tuning set of different files (stated in Section IV), never on the test files. The chosen values changed mean bits per byte by under 1 % from the next best, so the result is not sensitive to them; however, the best counter limit (30) was the smallest value tried, so a slightly better setting may exist.

E. When does coding win on the wire?

Let S be the file size, B the bandwidth in bytes per second, r the compression ratio, and V_c, V_d the codec's compress and decompress throughput in bytes of original data per second. Sending raw takes S/B. We consider three cases. In the *static* case the sender compressed the file in advance, so only the transfer and decompression are on the clock: $T = rS/B + S/V_d$. In the *sequential* case compression, transfer and decompression run one after another. In the *pipelined* case chunked streaming overlaps all three, an optimistic idealization where the slowest stage sets the time. Coding is faster than raw exactly when

$$B < (1-r)V_d, \quad B < \frac{1-r}{1/V_c + 1/V_d}, \quad B < \min(V_c, V_d) \tag{1}$$

respectively (the third requires $r < 1$). Round-trip time and file size cancel. For two codecs, a is faster than c in the static case when

$$V_{d,a} > \frac{1}{(r_c - r_a)/B + 1/V_{d,c}}, \tag{2}$$

which gives the decoding speed a stronger but slower codec must reach to be worth using at a given bandwidth. The model ignores TCP dynamics, loss, parallel connections and CPU contention, and uses throughput measured on one machine.

IV. Experimental Setup

Corpus. Table I lists the ten test files, all real except one control. They come from the test machine (Python standard library, a system log, a compiled binary, JSON metadata, a JPEG) and from public test data (a bacterial genome and a viral genome). Files are capped at 1 MiB. A separate tuning set, used only to choose augur-cm parameters, contains different files of the same kinds: odd-indexed standard-library sources, license text, an apt log, other JSON, a slice of another binary and a plasmid sequence. A manifest with SHA-256 hashes accompanies the corpus builder, because machine-local files differ between machines.

TABLE I
Test corpus (bytes). Three files are incompressible by construction or by prior compression.

File	Bytes	Source
source code	1,048,576	Python stdlib `*.py`, even-indexed
prose (docs)	563,555	Python language reference text
system log	683,440	`dpkg.log`
JSON	1,048,576	JSON metadata files
executable (ELF)	1,023,032	gcc driver binary
DNA, bacterial	1,048,576	*B. fragilis* chromosome, FASTA
DNA, viral	30,322	SARS-CoV-2 genome, FASTA
gzip file	1,048,576	gzip of the bacterial genome
JPEG image	292,349	a library image
random bytes	1,048,576	seeded generator (control)

Codecs. gzip level 9, bzip2 level 9, xz preset 9e, zstd level 19, brotli quality 11 with a 16 MiB window, and augur-cm. None uses a trained dictionary. Every output was decompressed and compared with the input by SHA-256, and the run aborts on any mismatch.

Measurement. Compress and decompress times are the median of three wall-clock runs, single-threaded, on one machine: an Intel Xeon at 2.10 GHz with two virtual CPUs and 7 GiB of memory, Linux 6.18, Python 3.13, gcc 13.3 at `-O2` with no hand-written SIMD. Throughput is bytes of original data per second. It is machine-specific and should be read as relative. augur-cm allocates 8×2^{22} four-byte slots (128 MiB) per run, a fixed cost that dominates its time on very small files.

Real-I/O runs. These used the author's workstation, not the virtual machine above: 16 logical CPUs, 61 GiB of memory, Linux 7.0, Python 3.14, and an ext4 file system on a drive the kernel reports as rotational (the model was not recorded). Five workloads (recurring routines, deep-learning epochs with fixed order and with reshuffling, sequential and random access) used 6000 accesses each, files of 125–750 MiB written fresh per workload, an application cache of 5 % of the unique bytes, 1 ms of compute per request, prefetch depth 4 with confidence threshold 0.4, and three repeats of seven configurations: 105 runs in all. The access orders are the synthetic ones of Section III-B.

V. Results

A. Compression

Table II and Fig. 2 give compressed bits per input byte. augur-cm produced the smallest output on six of the seven compressible files, from 6.8 % smaller than the best of the five general codecs on the viral genome to 31.5 % smaller on the system log, with 10.6 % on source code and 11.3 % on prose. On the executable, xz was 5.2 % smaller. The gains are largest where text-like structure repeats at distance, consistent with the ablation below. On the random file and the gzip file, augur-cm stored the data raw at a cost of one byte, as did the other codecs within a fraction of a percent. The JPEG shrank by 9.5 % with augur-cm and by 9.0 % with the best general

TABLE II
COMPRESSED BITS PER INPUT BYTE (LOWER IS BETTER; BEST IN BOLD). Δ SIZE COMPARES AUGUR-CM WITH THE BEST OF GZIP, BZIP2, XZ, ZSTD AND BROTLI (− MEANS SMALLER).

File	xz-9e	zstd-19	brotli-11	augur-cm	Δ size
DNA, bacterial	2.207	2.217	2.198	**2.044**	−7.0%
DNA, viral	2.382	2.261	2.214	**2.063**	−6.8%
executable (ELF)	**2.389**	2.555	2.446	2.512	+5.2%
JSON	0.711	0.739	0.692	**0.643**	−7.0%
prose (docs)	1.544	1.572	1.493	**1.325**	−11.3%
source code	1.627	1.656	1.602	**1.432**	−10.6%
system log	0.464	0.492	0.486	**0.315**	−31.5%
gzip file	8.001	8.000	8.000	**8.000**	–
JPEG image	7.358	7.364	7.279	**7.238**	–
random bytes	8.001	8.000	8.000	**8.000**	–

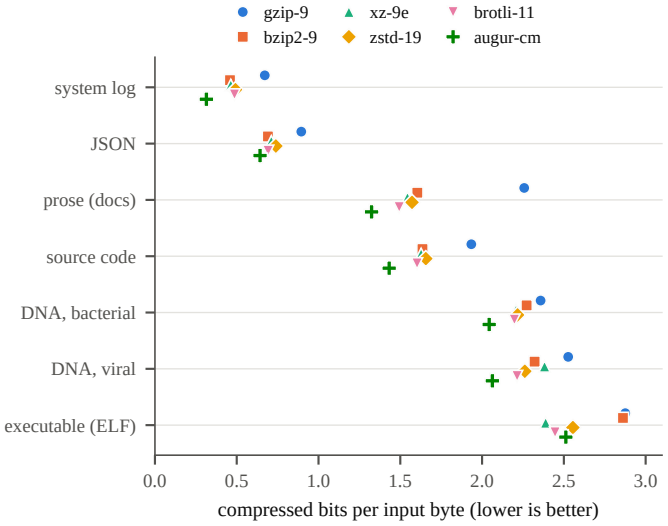


Fig. 2. Bits per byte by file type and codec (lower is better). Marker shape identifies the codec; rows are offset slightly to separate overlapping points. The three incompressible files are in Table II.

codec; we did not investigate whether this file is typical of JPEG data.

B. Speed

Table III summarizes the compressible files by median. augur-cm has the lowest median size and by far the lowest speed: about 0.5 MB/s in both directions, against 1090 MB/s for zstd decompression and 386 MB/s for brotli. Its compression speed is similar to brotli at quality 11, but brotli decompresses roughly 700 times faster. Context mixing is symmetric by nature, since the decoder repeats the encoder's work, while the strong dictionary codecs are asymmetric. On the 30 kB viral genome augur-cm ran at only 0.24 MB/s because of its fixed set-up cost.

C. Time on the wire

Applying Eq. (1) to the measured ratio and speed gives Table IV. Against sending raw, augur-cm wins on a static file below 2.7–5.9 Mbit/s, so it does beat no compression on slow links. The comparison that matters is against the other codecs.

TABLE III
MEDIAN OVER THE SEVEN COMPRESSIBLE FILES.

Codec	bits/byte	comp. MB/s	decomp. MB/s
gzip-9	2.258	6.82	257
bzip2-9	1.636	11.7	31.4
xz-9e	1.627	2.32	74.4
zstd-19	1.656	2.49	1090
brotli-11	1.602	0.50	386
augur-cm	1.432	0.52	0.52

TABLE IV
BANDWIDTH (MBIT/S) BELOW WHICH AUGUR-CM BEATS SENDING RAW, BY SCENARIO, DERIVED FROM MEASURED RATIO AND SPEED (1 MIB FILES; THE 30 KB FILE IS EXCLUDED BECAUSE SET-UP COST DOMINATES IT).

File	static	sequential	pipelined
DNA, bacterial	3.08	1.54	4.14
executable (ELF)	2.73	1.37	3.98
JSON	5.25	2.66	5.71
prose (docs)	3.27	1.64	3.92
source code	3.61	1.76	4.20
system log	5.88	2.96	6.13

Fig. 3 shows the time to deliver 1 MiB of source code: augur-cm is the fastest option only on very slow links, then flattens at about two seconds, the cost of decoding. Table V lists the fastest option per file type; at 0.5 Mbit/s and above, augur-cm is never first. Its crossover (Table VI) lies at 0.035–0.11 Mbit/s on five of the six file types, and on the executable it never wins. To be the fastest option at 2 Mbit/s, its decoding would have to be 16–53 times faster (Eq. 2), 72–214 times at 10 Mbit/s and 321–822 times at 50 Mbit/s. The advantage in bytes is real; the advantage in time is not, with this implementation, except under very scarce bandwidth.

D. Ablation

Table VII varies the model. Removing the match model costs 4.0–20.6 % on every non-DNA file, so it is the most valuable single component. Restricting contexts to orders 1–4 costs 0.3–3.9 % and more than doubles speed (1.18 versus 0.53 MB/s); a single order-2 context costs 3.2–24.5 % but runs nearly four times faster. Orders 8 and 12 add up to 2.4 %. Table size matters: 2^{16} slots costs 6.1–10.5 %, while 2^{24} slots gains only 0.5–1.0 % at four times the memory (512 MiB) and less than half the speed. DNA is nearly unaffected by every variant (within 0.3 %), which shows that byte-oriented contexts do not exploit its four-letter structure; a model built for that alphabet would likely do better, but we did not test one.

E. Prefetching in simulated storage

Table VIII shows end-to-end speedup over an LRU cache without prefetching on the SATA SSD model, with the cache holding 5 % of the data, medians over three seeds. These results are simulated with nominal device parameters; they are hypotheses that Section V-F tests, not measurements. Conventional readahead already captures sequential access (1.38×), and the oracle shows that headroom exists elsewhere (1.28–1.50×). A history-based Markov predictor gains only

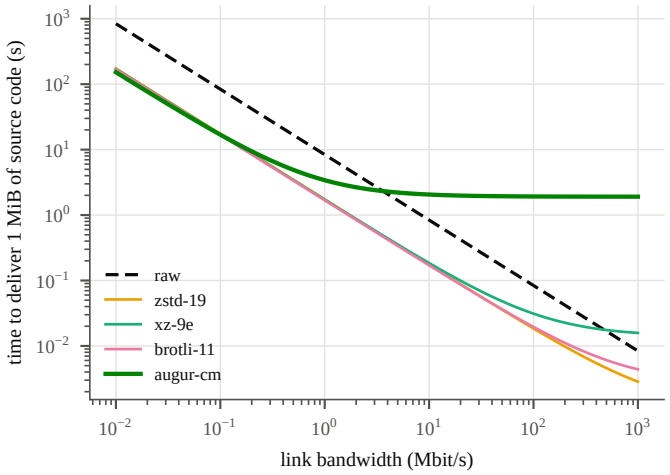


Fig. 3. Static-case time to deliver 1 MiB of source code versus link bandwidth, computed from measured ratios and speeds. Below about 0.1 Mbit/s augur-cm is fastest; above it the other codecs win because they decode far faster.

TABLE V
FASTEST OPTION (INCLUDING RAW) BY LINK SPEED IN MBIT/S, STATIC CASE.

File	0.5	2	10	50	250
DNA, bacterial	brotli-11	brotli-11	zstd-19	zstd-19	zstd-19
executable (ELF)	xz-9e	xz-9e	brotli-11	zstd-19	zstd-19
JSON	brotli-11	brotli-11	brotli-11	zstd-19	zstd-19
prose (docs)	brotli-11	brotli-11	brotli-11	zstd-19	zstd-19
source code	brotli-11	brotli-11	brotli-11	zstd-19	zstd-19
system log	bzip2-9	xz-9e	brotli-11	zstd-19	zstd-19

where history repeats: 1.08× on recurring routines (1.10× with the back-off gate) and 1.09× on fixed-order epochs. On random and reshuffled access it learns nothing, and accuracy does not equal speed: on the routines workload it reached 85 % accuracy but its tail latency was worse than the baseline because wasted prefetches queue ahead of real requests. Trusting a single observation is dangerous: the naive variant fell to 0.52× on random and 0.36× on reshuffled data, which is why the default requires two observations and smooths confidence, at the cost of learning more slowly (fixed-order epochs: 1.09× versus 1.19× over only three epochs). The outcome also depends on device speed relative to compute: with 100 μs of compute per request, the Markov predictor on the routines workload gives 1.40× on the NVMe model but 0.89× on the HDD model, where reads are too slow for overlap to help and extra reads add contention.

F. Prefetching on a real drive

The cold start worked: random reads of a cold file took 8.5–11.1 ms against 11–19 μs warm (ratios of 592–983), the signature of a seek-bound disk. Table IX gives end-to-end speedup over the LRU application cache; 7 of the 105 runs were disturbed by other activity on the machine and are excluded from the pairing (Section III-C). Table X gives details for the two workloads where history can be learned.

TABLE VI
HIGHEST BANDWIDTH AT WHICH AUGUR-CM IS STILL THE FASTEST OPTION (STATIC CASE), AND THE FACTOR BY WHICH ITS DECODING MUST SPEED UP TO BE FASTEST AT 2, 10 AND 50 MBIT/S (EQ. 2).

File	fastest below	speed-up needed at (Mbit/s)		
	(Mbit/s)	2	10	50
DNA, bacterial	0.079	22×	106×	442×
executable (ELF)	never	–	–	822×
JSON	0.035	53×	214×	570×
prose (docs)	0.081	23×	105×	348×
source code	0.093	21×	93×	329×
system log	0.11	16×	72×	321×

TABLE VII
ABLATION OF AUGUR-CM. Δ IS THE CHANGE IN COMPRESSED SIZE RELATIVE TO THE FULL MODEL, AS THE MEAN AND RANGE OVER THE FIVE NON-DNA FILES (SOURCE CODE, PROSE, LOG, JSON, EXECUTABLE); DNA IS THE BACTERIAL GENOME. MB/S IS THE MEDIAN COMPRESS THROUGHPUT OVER THE FIVE FILES. PER-FILE VALUES ARE IN THE REPOSITORY.

Variant	mean Δ	range	DNA Δ	MB/s
full model	–	–	–	0.53
no-match-model	+12.9%	+4.0% to +20.6%	−0.1%	0.55
orders$\leq$4	+2.0%	+0.3% to +3.9%	−0.3%	1.18
orders$\leq$6	+1.1%	+0.0% to +2.4%	−0.2%	0.77
order-2-only	+12.5%	+3.2% to +24.5%	−0.2%	1.98
table-bits=16	+8.6%	+6.1% to +10.5%	+0.1%	1.53
table-bits=18	+4.3%	+3.7% to +5.2%	+0.0%	1.28
table-bits=20	+1.5%	+1.2% to +1.8%	+0.0%	0.83
table-bits=24	−0.7%	−1.0% to −0.5%	−0.1%	0.23

TABLE VIII
SIMULATED SPEEDUP OVER LRU WITHOUT PREFETCHING (SATA SSD MODEL, 3 SEEDS, MEDIAN).

Workload	sequential	stride	markov2	markov2+gate	markov2-naive	oracle
sequential	1.38	1.38	1.00	1.00	1.00	1.38
random	1.00	1.00	1.00	1.00	0.52	1.28
Zipf working set	1.00	1.00	1.00	1.00	0.73	1.50
phased	1.06	1.13	0.99	1.02	0.69	1.30
recurring routines	1.00	1.00	1.08	1.10	1.05	1.28
DL, reshuffled	1.00	1.00	1.00	1.00	0.36	1.31
DL, fixed order	1.00	1.00	1.09	1.09	1.19	1.31

TABLE IX
MEASURED SPEEDUP OVER THE LRU APPLICATION CACHE ON A ROTATIONAL DRIVE, COLD CACHE AT EVERY RUN (MEDIAN OF PAIRED, UNDISTURBED REPEATS; ABOVE 1 IS FASTER). OS: KERNEL ONLY, WITH READAHEAD (RA) ON OR OFF; SEQ.: SEQUENTIAL PREFETCH; +GATE: MARKOV2 WITH THE BACK-OFF GATE. *: FEWER THAN THREE UNDISTURBED PAIRS.

Workload	OS	OS, no RA	seq.	markov2	+gate	oracle
recurring routines	0.98*	0.93	0.97*	0.94	0.97	1.30*
DL, fixed order	0.99*	0.81*	1.01*	1.02*	1.03*	1.21*
DL, reshuffled	0.99	0.82	1.00	1.00	1.02	1.21
sequential	0.98	0.83	0.98	0.99	0.97	0.94
random	0.99	0.99	1.01	0.99	0.99	1.31

A learned predictor did not pay on this drive. The Markov predictor ran at 0.94× on recurring routines, slower than LRU in all three repeats: it hit 34 % of requests but only 57 % of its prefetches were used, so it read 28 % more data from the device (981 versus 766 MiB) and its P99 latency rose from 35

TABLE X
DETAILS FOR THE TWO WORKLOADS WHERE HISTORY CAN BE LEARNED. HIT, ACCURACY, WASTED PREFETCH BYTES, BYTES THE KERNEL READ FROM THE DEVICE, AND P99 LATENCY ARE MEDIANS OVER REPEATS; TIME IS THE MEDIAN OF UNDISTURBED RUNS.

Workload	config	hit (%)	acc. (%)	wasted (MiB)	read (MiB)	P99 (ms)	time (s)
recurring routines	lru	1	–	0.0	766	35.4	59.8
	markov2	34	57	190.6	981	53.6	63.2
	markov2+gate	23	77	49.8	820	41.4	61.6
	oracle	100	100	0.0	792	51.6	46.3
DL, fixed order	lru	0	–	0.0	691	40.1	53.2
	markov2	33	100	0.3	691	41.3	52.1
	markov2+gate	33	100	0.3	691	40.7	51.7
	oracle	100	100	0.0	700	50.5	44.0

to 54 ms. The confidence gate cut wasted reads from 191 to 50 MiB and the loss to 0.97×, but it was still slower in every repeat. On fixed-order epochs the predictor was always right, yet it gained only 1.02–1.03×, because it must see a transition twice and therefore predicts only in the third of three epochs (hit rate 33 %). On reshuffled and random access it learned nothing and changed nothing (0.99–1.02×).

A perfect predictor would help, to a limit. The clairvoyant oracle reached 1.21–1.31× on the four non-sequential workloads (1.30× on routines), so the information is worth having, but no more: this drive's random-read rate is the ceiling, with prefetch depth 4 and four threads. Deeper queues might raise it; we did not test that. Its P99 latency is higher than LRU's, because requests that arrive before their prefetch completes wait for it.

Accuracy is not speed, and the kernel already prefetches. On the sequential workload the sequential prefetcher hit every request with 100 % accuracy and still ran at 0.98×, because kernel readahead already serves that pattern. Switching readahead off made the sequential and image workloads 0.81–0.83× as fast as with it on.

Prediction itself was cheap. Predictor time was at most 117 ms per run (at most 0.4 % of run time), process CPU time at most 7 % of wall time, and extra resident memory at most 38 MiB. The cost of the learned predictor was in the wasted reads it caused, not in running the model, although a neural model would change this.

What the simulator got wrong. The HDD simulation predicted the harm from the Markov predictor on recurring routines (0.89× simulated, 0.94× measured). It missed two effects: it gave the oracle no gain on routines (1.01×, because it models one FIFO queue, whereas the real drive gained 1.30× with several reads outstanding), and it gave sequential readahead 1.11× where the kernel's own readahead leaves nothing to gain.

VI. DISCUSSION

What the results support. Prediction reliably saves bytes on predictable data, and a small model with no deep neural network already beats strong general codecs by 7–31 % on logs, text, code, JSON and DNA. It saves nothing on random or already-compressed data, and the fallback keeps the cost to one byte. Saving time is a separate question that depends on compute speed against link speed: with an unoptimized single-threaded model the answer is no above roughly 0.1 Mbit/s. This also bears on the original motivation of helping people without fast connections: a slow link favors heavy compression, but the receiving device must run the model, so decode speed rather than model quality is the limiting factor. Access prediction showed the same shape on a real disk: the information is worth up to about 1.2–1.3×, a learned history model captured almost none of it, and it lost when its guesses were noisy, because wrong prefetches cost seeks.

What could change the answer. Decoding would need to be 16–53 times faster to win at 2 Mbit/s. Options we did not test are optimized and vectorized code, a GPU, splitting data into independently coded chunks to use several cores (at some cost in ratio), smaller or quantized neural predictors, and pretrained models shipped once and reused. Where bandwidth is metered rather than time-limited, the byte saving alone has value. For prefetching, longer training runs (a fixed-order predictor learns only after two epochs, so its benefit should grow with the epoch count), real access traces, faster drives, deeper prefetch queues, and a better-calibrated predictor are untested. Server-side prediction paired with a client budget hint may avoid the waste seen in HTTP/2 push [11], but we did not build or test it.

Threats to validity. (i) All compression timing is from one shared virtual machine, three runs per cell; speed values are relative and ratios are unaffected. (ii) augur-cm is a small, unoptimized model; we did not run PAQ8, cmix or NNCP themselves, so we make no claim against the state of the art. (iii) Corpus files are small (at most 1 MiB), include machine-local files, and cover few types; gains on other data may differ. (iv) Parameters were tuned on separate files, but all zstd and brotli settings are defaults for their levels and no dictionaries were trained, which would help small files. (v) The wire model is closed-form and ignores TCP and loss. (vi) The real-I/O results come from one machine and one rotational drive of unrecorded model, with three repeats; the access orders are synthetic; the kernel page cache is emulated as bounded rather than limited by memory pressure; and 7 of 105 runs were disturbed and handled by the pairing rule above. An SSD or NVMe drive would change the picture. (vii) The learned (neural) predictor of the original plan has not been built; only Markov and heuristic predictors were evaluated, so we make no claim about neural prefetching.

VII. CONCLUSION

A model that predicts data can make it smaller, often substantially, and exactly. Whether that makes it faster depends on how cheap the prediction is relative to the time saved. In our measurements, a context-mixing codec beat five standard codecs on size for text-like data but would need decoding 16–53 times faster to win on time at 2 Mbit/s. A learned access predictor, tested with real reads on a hard disk, gained at most a few percent where the access order repeated exactly and lost

where its guesses were noisy, while a perfect predictor would have gained 1.21–1.31×; the simulator had been right about the harm and wrong about the ceiling. Next steps are real access traces and solid-state drives, an optimized and parallel codec, a small neural byte predictor, and a network study of server-side prediction with client feedback.

Availability

Code, corpus builder with hashes, raw measurements, and this paper's source are at https://github.com/CygnusXl/Augur (version 0.3.0).

Acknowledgment

Experiment design, code and drafting were carried out with substantial assistance from Claude, an AI assistant from Anthropic. The author is responsible for the content.

References

[1] V. N. Padmanabhan and J. C. Mogul, "Using predictive prefetching to improve World Wide Web latency," *ACM SIGCOMM Computer Communication Review*, vol. 26, no. 3, pp. 22–36, 1996.

[2] J. Yang, R. Karimi, T. Sæmundsson, A. Wildani, and Y. Vigfusson, "MITHRIL: Mining sporadic associations for cache prefetching," in *Proc. ACM Symposium on Cloud Computing (SoCC)*, 2017.

[3] N. Dryden, R. Böhringer, T. Ben-Nun, and T. Hoefler, "Clairvoyant prefetching for distributed machine learning I/O," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[4] J. Mohan, A. Phanishayee, A. Raniwala, and V. Chidambaram, "Analyzing and mitigating data stalls in DNN training," *Proc. VLDB Endowment*, vol. 14, no. 5, 2021.

[5] G. Leclerc, A. Ilyas, L. Engstrom, S. M. Park, H. Salman, and A. Madry, "FFCV: Accelerating training by removing data bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023.

[6] Z. Song, D. S. Berger, K. Li, and W. Lloyd, "Learning relaxed Belady for content distribution network caching," in *Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, pp. 529–544.

[7] D. L.-K. Wong, H. Wu, C. Molder, S. Gunasekar, J. Lu, S. Khandkar, A. Sharma, D. S. Berger, N. Beckmann, and G. R. Ganger, "Baleen: ML admission & prefetching for flash caches," in *Proc. USENIX Conf. File and Storage Technologies (FAST)*, 2024.

[8] Z. Shi, A. Jain, K. Swersky, M. Hashemi, P. Ranganathan, and C. Lin, "A hierarchical neural model of data prefetching," in *Proc. Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.

[9] R. Bera, K. Kanellopoulos, A. V. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, "Pythia: A customizable hardware prefetching framework using online reinforcement learning," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2021.

[10] I. U. Akgun, A. S. Aydin, A. Burford, M. McNeill, M. Arkhangelskiy, A. Shaikh, L. Velikov, and E. Zadok, "KML: Using machine learning to improve storage systems," arXiv:2111.11554, 2021.

[11] Chromium project, "Intent to remove: HTTP/2 and gQUIC server push," blink-dev mailing list, https://groups.google.com/a/chromium.org/g/blink-dev/c/K3rYLvmQUBY, accessed Oct. 2026.

[12] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[13] I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic coding for data compression," *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, 1987.

[14] M. V. Mahoney, "Adaptive weighing of context models for lossless data compression," Florida Institute of Technology, Computer Science Dept., Tech. Rep. CS-2005-16, 2005.

[15] F. Bellard, "NNCP: Lossless data compression with neural networks," https://bellard.org/nncp/, accessed Oct. 2026.

[16] G. Delétang, A. Ruoss, P.-A. Duquenne, E. Catt, T. Genewein, C. Mattern, J. Grau-Moya, L. K. Wenliang, M. Aitchison, L. Orseau, M. Hutter, and J. Veness, "Language modeling is compression," arXiv:2309.10668, 2023.

[17] Y. Collet and E. M. Kucherawy, "Zstandard compression and the application/zstd media type," IETF, RFC 8878, 2021.

[18] J. Alakuijala and Z. Szabadka, "Brotli compressed data format," IETF, RFC 7932, 2016.